Fiche d'exercices — Fonctions et boucles en Python

1re NSI

Nom : ..... Prénom : ..... Classe : .....

Chaque cadre reproduit la fenêtre de **Pyzo** : à gauche l'**éditeur** (le script), à droite le **shell** (les appels de fonctions et les affichages). Les réponses s'écrivent directement sur les pointillés.

Partie A — Les fonctions

Exercice 1 — Lire du code

Les trois fonctions ci-dessous ont été exécutées. Compléter ce que renvoie le shell.

▸ script.py

```
def double(x):
    return x * 2

def diviser(a, b):
    return a / b

def repete(mot):
    return mot + mot
```

▸ Shell Python

```
>>> double(5)
.....
>>> double(-3)
.....
>>> diviser(10, 2)
.....
>>> repete("bon")
.....
>>>  .....
20.0
```

Pourquoi `diviser(10, 2)` ne renvoie-t-il pas 5 ? .....

Exercice 2 — Fonctions mathématiques

Compléter les fonctions $f(x) = 2x + 5$, $F = C \times \frac{9}{5} + 32$ (conversion °C → °F) et $g(x) = x^2 - 3x + 1$.

▸ script.py

```
def f(x):
    return ..........................

def celsius_vers_fahrenheit(c):
    return ..........................

def g(x):
    return ..........................
```

▸ Shell Python

```
>>> f(3)
11
>>> celsius_vers_fahrenheit(0)
32.0
>>> celsius_vers_fahrenheit(100)
.....
>>> g(2)
-1
```

Exercice 3 — Fonctions et chaînes de caractères

▸ script.py

```
def bonjour(prenom):
    return ..........................

def triple_mot(mot):
    return ..........................

def encadre(texte):
    return ..........................
```

▸ Shell Python

```
>>> bonjour("Alice")
'Bonjour Alice'
>>> triple_mot("ha")
'hahaha'
>>> encadre("Python")
'[Python]'
>>> encadre(triple_mot("o"))
.....
```

Exercice 4 — Fonctions avec conditions

Rappel : `n % 2` est le reste de la division de `n` par 2. La valeur absolue de x vaut x si $x \geq 0$ et $-x$ sinon.

▸ script.py	▸ Shell Python

`def est_pair(n):`
`    return ..........`
 	
`def valeur_absolue(x):`
`    if .......... :`
`        return ..........`
`    else:`
`        return ..........`
 	
`def maximum(a, b):`
`    ..........`
`    ..........`
`    ..........`
`   ....
	`>>> est_pair(4)`
	`True`
	`>>> est_pair(7)`
	`..........`
	`>>> valeur_absolue(-5)`
	`5`
	`>>> maximum(5, 8)`
	`8`
	`>>> maximum(-2, -9)`
	`..........

Bonus — année bissextile. Une année est bissextile si elle est divisible par 4, sauf si elle est divisible par 100 (elle doit alors aussi être divisible par 400).

▸ script.py	▸ Shell Python

`def est_bissextile(annee):`
`    ..........`
`   ....
	`>>> est_bissextile(2024)`
	`True`
	`>>> est_bissextile(1900)`
	`False`
	`>>> est_bissextile(2000)`
	`True`

Partie B — Les boucles

Exercice 5 — Valeurs prises par le variant

Donner la séquence de valeurs prise par `i`.

Instruction	Séquence de valeurs
`for i in range(4):` *(exemple)*	`0, 1, 2, 3`
`for i in range(10):`	....
`for i in range(4, 10):`	....
`for i in "abacab":`	....
`for i in [6, 4, 2]:`	....

Exercice 6 — Compléter l'itérable

Compléter la boucle pour obtenir l'affichage du shell.

▸ script.py	Shell

`for i in ........:`
`    print(i)`
	`0`
	`1`
	`2`

▸ script.py	Shell

`for i in ........:`
`    print(i)`
	`T`
	`G`
	`V`

▸ script.py	Shell
`for i in ........:` `    print(i)`	`2001` `2011`

Exercice 7 — Prévoir l'affichage

Écrire dans le shell ce qu'affiche chaque script.

▸ script.py	Shell

`for i in range(4):`
`    if i > 2:`
`        print(i)`
	.....
	.....

▸ script.py	Shell

`for j in range(4):`
`    if j % 2 == 0:`
`        print(j)`
	.....
	.....

▸ script.py	Shell

`L = [2012, 2019, 2024]`
`for k in L:`
`    if k % 3 == 0:`
`        print(k)`
	.....
	.....

Exercice 8 — Condition d'arrêt d'une boucle `while`

Instruction	Condition d'arrêt
`while i < 4:` *(exemple)*	`i >= 4`
`while i > 10:`	.....
`while i <= 3:`	.....
`while i > j and i != 0:`	.....

Exercice 9 — De `for` à `while`

Réécrire le script de gauche avec une boucle `while`. *Rappel : `for i in range(10):` équivaut à `i = 0` puis `while i < 10:` … sans oublier `i = i + 1` à la fin du bloc.*

▸ script_for.py	▸ script_while.py

`for j in range(4):`
`    if j % 2 == 0:`
`        print(j)`	

Exercice 10 — Variant de boucle et terminaison

Pour chaque script, donner le variant de boucle et dire si la boucle se termine.

```
i = 0
a = 10
while i < 3:
    i = i + 1
    a = 10 * i
```

Variant : .....
Termine : oui / non

```
a = 4
b = 10
while a > 3:
    a = a - 1
    b = b + 1
```

Variant : .....
Termine : oui / non

```
i = 0
a = 10
while a > 3:
    i = i + 1
    a = a + 10
```

Variant : .....
Termine : oui / non

Partie C — Fonctions et boucles

Exercice 11 — Traitements automatisés sur des listes

Rappels : `chr(110)` donne le caractère de rang 110 dans la table ASCII, `ord("n")` donne le rang du caractère `"n"`. `L.append(x)` ajoute `x` à la fin de la liste `L`.

▸ script.py

```
def somme(L):
    s = 0
    for valeur in L:
        s += .........
    return .........

def decode(L):
    chaine = ""
    for rang in L:
        chaine += chr(.........)
    return chaine

def encode(message):
    L = []
    for c in message:
        L.append(.........)
    return L
```

▸ Shell Python

```
>>> somme([65400, 31654, 1026])
.....
>>> decode([110, 99, 111])
.....
>>> encode("abacab")
.....
```

Exercice 12 — Multiplication par additions successives

Multiplier `a` par `b` revient à calculer `a + a + … + a` (`b` fois). La variable `produit` stocke les additions successives et `b` sert de compteur : on le diminue de 1 à chaque itération.

1. Compléter la fonction (seuls `+` et `-` sont autorisés).

▸ script.py

```
def multiplier(a, b):
    produit = 0
    while b > .....:
        produit = produit + a
        b = b - .....
    return produit
```

▸ Shell Python

```
>>> multiplier(3, 8)
.....
```

2. Compléter le tableau de suivi des variables pour l'appel `multiplier(3, 8)`.

b avant le test	**8**								
test `b > 0`									
`produit` après le tour									

3. Justifier que la fonction se termine, à l'aide du tableau.

.....

.....

Exercice 13 — Division entière par soustractions successives

Écrire la fonction `division(a, b)` qui renvoie le quotient entier de `a` par `b`, en n'utilisant que l'opérateur `-` (et `+` pour compter).

▸ script.py

```
def division(a, b):
```

▸ Shell Python

```
>>> division(17, 5)
3
>>> division(20, 4)
.....
>>> division(3, 7)
.....
```